

Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

Tarea: Estructura de Datos

Santiago Adrian Lazos Ovalle

ID de Estudiante: 202522695

Materia: Estructura de Datos y sus Aplicaciones

Fecha de entrega: 29 de agosto de 2026

Periodo: Otoño 2026

Repositorio con la implementación completa del proyecto: [Estructura de Datos/Pila en GitHub](#).

Descripción breve del funcionamiento del algoritmo

El programa inicia recibiendo la expresión matemática ingresada por el usuario y calcula su longitud. Con este dato, se instala un objeto de la clase Pila con la capacidad exacta de la cadena, asegurando así un uso dinámico y eficiente de la memoria. Se utiliza también una bandera lógica (Error_de_sintaxis) inicializada en falso para llevar el control del estado de la evaluación.

Posteriormente, el algoritmo recorre la expresión carácter por carácter de izquierda a derecha mediante un ciclo. Se implementó una lógica de filtrado para que el programa ignore números, letras y operadores aritméticos, enfocando el análisis exclusivamente en los delimitadores.

El funcionamiento central se divide en dos acciones principales:

1. Aperturas: Cada vez que el ciclo detecta un delimitador de apertura ((, [, {}), lo almacena inmediatamente en la cima de la pila mediante la operación apilar().
2. Cierres: Si el ciclo detecta un delimitador de cierre (),], }), primero valida que la pila no esté vacía (lo cual indicaría un cierre sin apertura). Si la pila contiene elementos, se obtiene la cima actual y se compara para comprobar si hace la pareja correcta con el cierre detectado. Si son compatibles, se retira el elemento con desapilar(); de lo contrario, se enciende la bandera de error y se rompe el ciclo para detener el análisis.

Finalmente, al concluir el recorrido de toda la cadena, el programa emite su veredicto. La expresión solo se considera sintácticamente válida si la pila quedó completamente vacía (comprobando que todo lo que se abrió se cerró) y si la bandera de error nunca se activó.

Código Fuente:

```
from Pila import *

expresion = input("Ingresa la expresion a evaluar: ")
z=len(expresion)
Pilaespecial= Pila(z)
Error_de_sintaxis=False

if z==0:
    print(f"Error no hay expresion")

else:
    for caracter in expresion:

        if caracter == "{" or caracter == "[" or caracter== "(":
            Pilaespecial.apilar(caracter)

        elif caracter == "}" or caracter == "]" or caracter== ")":

            if Pilaespecial.esVacio():
                print("Error Revisa (Cierre sin apertura): ",caracter)
                Error_de_sintaxis=True
                break

            cima=Pilaespecial.ObtenerCima()

            if caracter == ")" and cima == "(" or caracter == "]" and cima == "["
or caracter == "}" and cima == "{":
                Pilaespecial.desapilar()

            else:

                if caracter == ")" or caracter == "}" or caracter == "]":
                    print("Error Revisa (Apertura sin cierre): ",caracter)
                    Error_de_sintaxis=True
                    break

            if Pilaespecial.esVacio() and Error_de_sintaxis==False:
                print("La expresion es correcta")

        else:
            print("La expresion es incorrecta")
```

Clase Pila:

```
class Pila:#Con esta Clase voy a crear una Pila de x tamaño y realizar sus metodos

    def __init__(self, capacidad):#Esta parte es el constructor
        self.capacidad = capacidad
        self.cima = -1
        self.arreglo = [None] * self.capacidad

    def esVacio(self):#Compara la cima de la pila con -1 para decir si esta vacia o no
        if self.cima == -1:
            return True
        else:
            return False

    def esLleno(self):#Compara la cima con la capacidad para checar si esta llena
        if self.cima == (self.capacidad-1):
            return True
        else:
            return False

    def mostrarPila(self):
        if self.esVacio():
            print(f"Esta Pila Esta Vacía")
        else:
            print("=====Pila=====")
            for i in range(self.cima, -1, -1):#Empieza en la cima, luego nos dice el final es -1, y
va de -1 en -1
                print(self.arreglo[i])
            print("=====Fin_Pila=====")

    def apilar(self, elemento):
        if self.esLleno() == True:
            print(f"Esta Pila esra llena, No se pueden agregar elementos")

        else:
            self.cima += 1
            self.arreglo[self.cima] = elemento

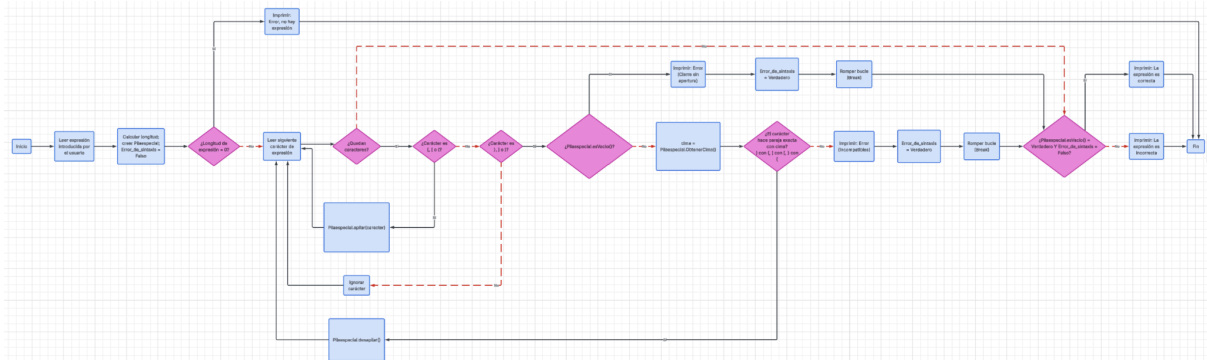
    def desapilar(self):
        if self.esVacio() == True:
            print(f"Esta Pila Esta Vacía, No se pueden eliminar elementos")
        else:
            self.cima -= 1
            return self.arreglo[self.cima + 1]

    def verCIMA(self):
        if self.esVacio():
            print(f"Esta Pila esra Vacía")
        else:
            print(self.arreglo[self.cima])

    def ObtenerCima(self):
        if self.esVacio():
            return False
        else:
            return self.arreglo[self.cima]
```

Diagrama de Flujo en Lucidchart

Ya con el proyecto terminado procedí a realizar el diagrama de flujo del programa: “Analizador Sintáctico”. [Lucidchart](#), en ese link se encuentra el diagrama permitiendo así una mejor visualización.



Casos de Prueba

Caso 1 (Expresión válida): $(a+b)^*[c-d]$

Al ejecutar esta expresión este es el resultado

```

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code> & C:\Program Files\dotnet\dotnet.exe run
Ingresa la expresion a evaluar: (a+b)*[c-d]
La expresion es correcta
PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code>

```

Caso 2 (Expresión válida con diversos niveles de agrupación): $\{[(a+b)*c]-d\}$

Al ejecutar esta expresión este es el resultado

```

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code> &
Ingresa la expresion a evaluar: {[ (a+b)*c ]-d}
La expresion es correcta
PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code>

```

Caso 3 (Delimitadores incompatibles): (a+b]

Al ejecutar esta expresión este es el resultado

```
PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code> &
Ingresa la expresion a evaluar: (a+b]
Error Revisa: ]
La expresion es incorrecta
PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code>
```

Caso 4 (Delimitador de cierre sin apertura): (a+b))

Al ejecutar esta expresión este es el resultado

```
PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code> &
Ingresa la expresion a evaluar: (a+b))
Error Revisa (Cierre sin apertura): )
La expresion es incorrecta
PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code>
```

Caso 5 (Delimitador de apertura sin cierre): ((a+b)

Al ejecutar esta expresión este es el resultado

```
PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code> &
Ingresa la expresion a evaluar: ((a+b)
La expresion es incorrecta
PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code>
```

Caso 6 (Anidamiento / Orden incorrecto): {[(a+b}]

Al ejecutar esta expresión este es el resultado

```
PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code> &
Ingresa la expresion a evaluar: {[ (a+b} ]
Error Revisa: }
La expresion es incorrecta
PS C:\Users\salez\AppData\Local\Programs\Microsoft VS Code>
```

Conclusión

Una pila es la estructura de datos más adecuada para resolver este problema porque su funcionamiento interno refleja exactamente la lógica del anidamiento matemático. Al leer una expresión, el último delimitador que abrimos (el más interno) es el que obligatoriamente debe cerrarse primero.

En cuanto a las operaciones, aunque apilar y desapilar son la base del movimiento de datos, la operación que resulta fundamental para que el análisis no falle es `ObtenerCima()`. Esta función es la clave lógica del programa, ya que nos permite "asomarnos" a ver cuál fue el último delimitador guardado y compararlo con el carácter de cierre actual, permitiéndonos tomar la decisión de desapilar solo si son compatibles, sin arruinar la estructura de los datos.

Finalmente, la relación entre el comportamiento LIFO (Last In, First Out) y el anidamiento de los delimitadores es directa y proporcional. El principio LIFO garantiza que el elemento más reciente en entrar sea el primero en salir, lo cual modela a la perfección la regla de sintaxis que dicta que las expresiones agrupadas deben resolverse siempre de adentro hacia afuera.